Continuiamo a vedere cosa sanno fare le TM.
Ma anche quello che NON sanno fare:

- Vedremo che esistono linguaggi NON TURING DECIDIBILI e nemmeno TURING RICONOSCIBILI.

Per ragionare con le TM un modo algoritmico spesso considereremo TM che prendono un input la codifica di un oggetto $\langle O \rangle$. Ovviamente, l'input è un reale sempre stringa; ma useremo le stringhe per codificare oggetti più complessi.

Es: Una TM può ricevere un input un grafo $\langle G \rangle$. Ad esempio quando le mostriamo

di decidere.

Vediamo esempi concreti: Decidere se DFA accetta una stringa data.

$$A_{DFA} = \{ \langle B, w \rangle : B \in DFA, w \in L(B) \}$$

(Spoiler: Se $B \in TM$, A_{TM} NON è DECIDIBILE!)

L'idea è molto:

- Su input $\langle B, w \rangle$ la TM deve simulare una esecuzione del DFA B su w.
- Se la simulazione termina in stato accettante la TM accetta; altrimenti rifiuta.

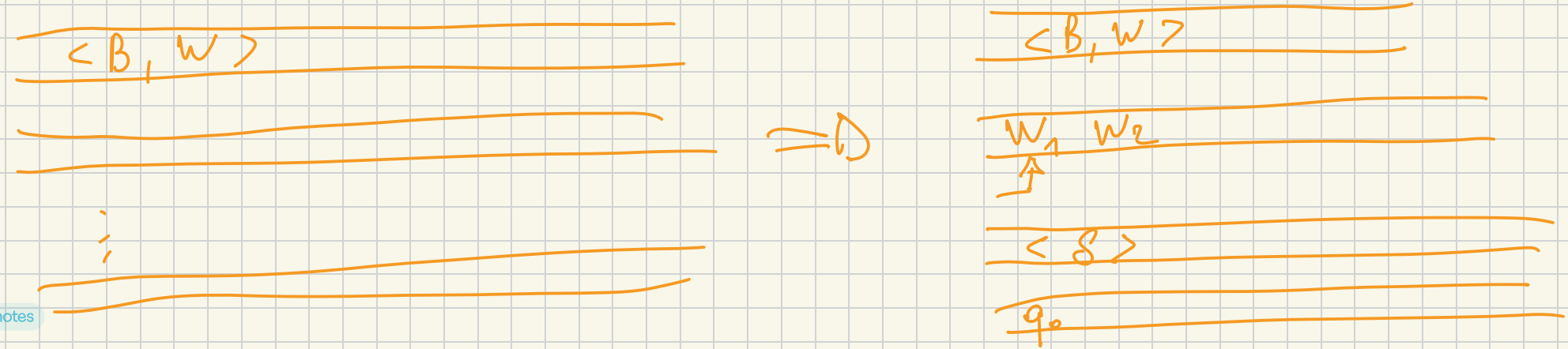
Made with Goodnotes

Implementazione ad alto livello:

- $B = (Q, \Sigma, \delta, q_0, F)$ è parte dell'input.

Posso immaginare di rappresentare la δ come sequenza di stringhe separate da "#".

- Per comodità posso anche pensare di avere la δ su un nastro apposito. Un nastro che memorizza solo w e un nastro per tenere traccia dello stato del DFA.



Made with Goodnotes

- Al t-simo passo di computazione: leggo stato del DFA, leggo carattere in input sul nastro contenente w, scorro la $\langle \delta \rangle$ per capire stato di arrivo e appiano lo stato su ultimo nastro.
- Accetto se e solo se dopo aver esaminato w lo stato nell'ultimo nastro appartiene ad F.

Altro esempio:

$$A_{NFA} = \{ \langle B, w \rangle : B \in NFA, \ w \in L(B) \}.$$

Idea: Definisco M' t.c. trasforma $\langle B, w \rangle$ in $\langle C, w \rangle$ dove $C \in DFA$ e $L(C) = L(B)$

Made with Goodnotes

come abbiamo visto a lezione.

Poi lancio M su $\langle C, w \rangle$.

Stessa cosa:

$$A_{REX} = \{ \langle R, w \rangle : R \in RE(\Sigma) \text{ e } w \text{ genera da } R \}$$

Vediamo un esempio diverso: Il test del vuoto.

$$E_{DFA} = \{ \langle B \rangle : B \in DFA, L(B) = \emptyset \}.$$

Idea: $L(B) \neq \emptyset$ se e solo se esiste un cammino che parte dallo stato iniziale e quello finale nel diagramma che rappresenta DFA.

Made with Goodnotes

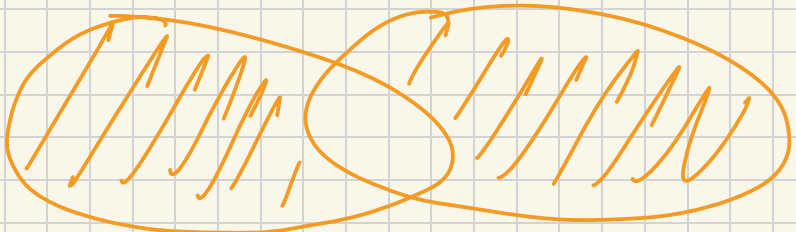
Posso usare algoritmo di marcatura:

- Marco lo stato q_0.
- Ripeto: Marco ∨ stato raggiungibile da stato già marcato.
- Controllo se uno degli stati in F è marcato. Se lo è rifiuto. Altrimenti accetto.

Posso pure decidere:

$$EQ_{DFA} = \{\langle B, B' \rangle : B, B' \in DFA, L(B) = L(B')\}.$$

Idea: Io posso "ridurre" al Test del vuoto!



Made with Goodnotes

$$L(B) \triangle L(B') = \left(L(B) \cap \overline{L(B')}\right) \cup \left(L(B') \cap \overline{L(B)}\right)$$

$$L(B) = L(B') \text{ sse } L(B) \triangle L(B') = \emptyset.$$

Quindi:

- Su input $\langle B, B' \rangle$ costruisco DFA B'' t.c. $L(B'') = L(B) \triangle L(B')$. Posso farlo perché REG è chiuso per $\cup, \cap, \overline{\ }$.
- Poi uso la TM per decidere se $\langle B' \rangle \in E_{DFA}$.

Made with Goodnotes

Vediamo che succede con le grammatiche:

$$A_{CFG} = \{ \langle G, w \rangle : G \in CFG, w \in L(G) \}.$$

Anche A_{CFG} è decidibile. $w \in L(G)$ se esiste derivazione; ma non posso provarle tutte. Potrei usare PDA. Oppure:

- Su input $\langle G, w \rangle$, trasformo G nella $\tilde{G}$ equivalente in forma normale.
- Abbiamo già visto che se $|w| = n$ allora $\tilde{G}$ genera w in al più $2n-1$ passi.
- Quindi la nostra TM deve provare solo le

derivazione di lunghezza al più $2n-1$.

Ultimo esempio:

$$E_{CFG} = \{ \langle G \rangle : G \text{ è CFG}, L(G) = \emptyset \}.$$

Posso usare ancora algoritmo di marcatura:

- Su input $\langle G \rangle$, marco tutti i terminali.
- Ripeto: Marco tutte le variabili A in G t.c. esiste regola $A \to U_1 \cdots U_k$ e $U_1 \cdots U_k$ sono marcati.
- Accetto sse S non è marcato.

Made with Goodnotes

Che succede con:

$$EQ_{CFG} = \{ \langle G, H \rangle : G, H \text{ CFG} \quad L(G) = L(H) \}?$$

Non è DECIDIBILE! Lo vedremo più avanti.

La situazione cambia se proviamo a considerare le TM:

$$A_{TM} = \{ \langle M, w \rangle : M \in TM, \ w \in L(M) \}.$$

NON È DECIDIBILE.

A_{TM} è TURING riconoscibile.

TEO
Made with Goodnotes

Dim. Idea di base; chiamiamo V la nostra TM:

- Su input $\langle M, w \rangle$, simula M su w
- Se durante la simulazione M accetta, accetto.
  Se rifiuta, rifiuto.

La TM V riconosce A_{TM}; ma non la decide perché M può andare in loop.
Se V avesse modo di capire se M va in loop o meno, V potrebbe diventare un decisore. Vedremo che questo è impossibile!